

Scikit-learn দিয়ে KNN (K-Nearest Neighbors)

মেশিন লার্নিং প্র্যাকটিক্যাল নোটস — সহজ বাংলায় ব্যাখ্যা

১. ভূমিকা

KNN অ্যালগরিদমের মূল ধারণা (দূরত্ব হিসাব, নিকটতম প্রতিবেশী বাছাই, ভোটগুণী) নিজে হাতে কোড করে বাস্তবায়ন করা সম্ভব হলেও, বাস্তব প্রজেক্টে আমরা সাধারণত **Scikit-learn** (**sklearn**) লাইব্রেরির **KNeighborsClassifier** ও **KNeighborsRegressor** ক্লাস ব্যবহার করি। এতে দূরত্ব হিসাব, প্রতিবেশী খোঁজা ও ভোটগুণীর সব জটিল কাজ আগে থেকেই অপটিমাইজড কোডে লেখা থাকে।

এই নোট আমারা দেখবে — **sklearn** দিয়ে কীভাবে ধাপে ধাপে একটি KNN মডেল তৈরি, ট্রেনিং, প্রেডিকশন ও মূল্যায়ন (evaluation) করতে হয়, সঠিক K বাছাই করার উপায়, এবং গুরুত্বপূর্ণ প্যারামিটার-মেথড নিয়ে আলোচনা করব।

২. প্রয়োজনীয় লাইব্রেরি ইম্পোর্ট করা

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import pandas as pd
```

এখানে **KNeighborsClassifier** হলো ক্লাসিফিকেশনের জন্য মূল মডেল ক্লাস (রিগ্রেশনের জন্য **KNeighborsRegressor** ব্যবহার করা হয়), বাকি লাইব্রেরিগুলো ডেটা ভাগ করা, স্কেলিং ও মূল্যায়নের জন্য ব্যবহৃত হয়।

৩. সম্পূর্ণ ওয়ার্কফ্লো (Step-by-Step)

ধাপ ১: ডেটা প্রস্তুত করা

```
# X = ইনপুট ফিচারসমূহ, y = আউটপুট লেবেল
X = df[['feature1', 'feature2', 'feature3']]
y = df['target']
```

ধাপ ২: ট্রেনিং ও টেস্ট ডেটা ভাগ করা

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

ধাপ ৩: ফিচার স্কেলিং করা (অত্যন্ত গুরুত্বপূর্ণ)

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

বিশেষভাবে গুরুত্বপূর্ণ: KNN সম্পূর্ণভাবে দূরত্বের হিসাবের উপর নির্ভরশীল, তাই লজিস্টিক রিগ্রেশনের তুলনায় এখানে ফিচার স্কেলিং করা আরও বেশি জরুরি। স্কেলিং না করলে বড় মানের ফিচার দূরত্বের হিসাবে অতিরিক্ত প্রভাব ফেলবে এবং মডেলের ফলাফল ভুল হতে পারে। এখানেও **fit_transform()** শুধু ট্রেনিং ডেটায়, আর **transform()** টেস্ট ডেটায় ব্যবহার করা হয়, যাতে ডেটা লিকাজ (data leakage) না হয়।

ধাপ ৪: মডেল তৈরি ও ট্রেনিং করা

```
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train_scaled, y_train)
```

n_neighbors=5 মানে K = ৫, অর্থাৎ নতুন ডেটা পয়েন্টের সবচেয়ে কাছের ৫টি প্রতিবেশী বিবেচনা করা হবে। মজার বিষয় হলো — যেহেতু KNN একটি "লেজি লার্নার", তাই **.fit()** মেথড আসলে তেমন কোনো ভারী গণনা করে না, শুধু ট্রেনিং ডেটা মেমরিতে সংরক্ষণ করে রাখে। প্রকৃত হিসাব-নিকাশ হয় প্রেডিকশনের সময়।

ধাপ ৫: প্রেডিকশন করা

```
y_pred = model.predict(X_test_scaled) # সরাসরি ক্লাস
y_pred_proba = model.predict_proba(X_test_scaled) # সম্ভাবনা
```

`.predict_proba()` -তে KNN প্রতিটি ক্লাসের সম্ভাবনা বের করে ঐ K-টি প্রতিবেশীর মধ্যে কতজন কোন ক্লাসে আছে তার অনুপাত হিসাব করে (যেমন ৫ জনের মধ্যে ৩ জন ক্লাস ১-এ থাকলে সম্ভাবনা হবে ০.৬)।

ধাপ ৬: মডেলের পারফরম্যান্স যাচাই করা

```
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Report:\n", classification_report(y_test, y_pred))
```

৪. সঠিক K-এর মান বাছাই করা (Elbow Method)

KNN-এ সবচেয়ে গুরুত্বপূর্ণ কাজ হলো সঠিক K বাছাই করা। এর জন্য সাধারণত বিভিন্ন K-এর মান পরীক্ষা করে দেখা হয়, কোনটিতে সবচেয়ে বেশি accuracy পাওয়া যায় — একে বলা হয় এলবো মেথড (Elbow Method)।

```
error_rates = []
for k in range(1, 21):
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_scaled, y_train)
    pred = knn.predict(X_test_scaled)
    error_rates.append(1 - accuracy_score(y_test, pred))

import matplotlib.pyplot as plt
plt.plot(range(1, 21), error_rates, marker='o')
plt.xlabel('K এর মান')
plt.ylabel('এরর রেট')
plt.show()
```

এই গ্রাফে যেখানে এরর রেট সবচেয়ে কম এবং স্থিতিশীল হয়ে যায় (গ্রাফের বাঁক বা "কনুই"-এর জায়গা), সেই K-এর মানই সাধারণত সবচেয়ে ভালো বাছাই।

৫. GridSearchCV দিয়ে স্বয়ংক্রিয়ভাবে সেরা K খোঁজা

ম্যানুয়ালি লুপ চালানোর বদলে `GridSearchCV` ব্যবহার করে ক্রস-ভ্যালিডেশনের মাধ্যমে স্বয়ংক্রিয়ভাবে সবচেয়ে ভালো প্যারামিটার খুঁজে বের করা যায়:

```
from sklearn.model_selection import GridSearchCV

param_grid = {'n_neighbors': range(1, 21)}
grid = GridSearchCV(KNeighborsClassifier(), param_grid, cv=5)
grid.fit(X_train_scaled, y_train)

print("সেরা K:", grid.best_params_)
print("সেরা Accuracy:", grid.best_score_)
```

৬. গুরুত্বপূর্ণ প্যারামিটার (Important Parameters)

প্যারামিটার	কাজ
<code>n_neighbors</code>	প্রতিবেশীর সংখ্যা (K) নির্ধারণ করে (ডিফল্ট = ৫)
<code>weights</code>	'uniform' (সব প্রতিবেশীর ভোট সমান) বা 'distance' (কাছের প্রতিবেশীকে বেশি গুরুত্ব)
<code>metric</code>	দূরত্ব পরিমাপের পদ্ধতি — 'minkowski' (ডিফল্ট), 'euclidean', 'manhattan'
<code>p</code>	Minkowski দূরত্বের প্যারামিটার — p=১ হলে Manhattan, p=২ হলে Euclidean দূরত্ব
<code>algorithm</code>	নিকটতম প্রতিবেশী খোঁজার পদ্ধতি — 'auto', 'ball_tree', 'kd_tree', 'brute'
<code>n_jobs</code>	সমান্তরালভাবে (parallel) কতগুলো CPU কোর ব্যবহার হবে (-1 দিলে সবগুলো ব্যবহার হয়)

উদাহরণ:

```
model = KNeighborsClassifier(
    n_neighbors=7,
    weights='distance',
    metric='euclidean'
)
```

এখানে `weights='distance'` ব্যবহার করায় যে প্রতিবেশীরা নতুন ডেটা পয়েন্টের সবচেয়ে কাছে থাকবে, ভোটাভুক্তিতে তাদের গুরুত্ব বেশি থাকবে (দূরের প্রতিবেশীর তুলনায়), যা অনেক ক্ষেত্রে ফলাফল আরও নির্ভুল করে।

৭. রিগ্রেসনের জন্য KNeighborsRegressor

যদি ক্লাস প্রেডিক্ট করার বদলে সংখ্যাগত মান (continuous value) প্রেডিক্ট করতে হয়, তাহলে ব্যবহার করা হয় `KNeighborsRegressor`, যার ব্যবহারবিধি প্রায় একই রকম:

```
from sklearn.neighbors import KNeighborsRegressor

reg_model = KNeighborsRegressor(n_neighbors=5)
reg_model.fit(X_train_scaled, y_train)
predictions = reg_model.predict(X_test_scaled)
```

এখানে প্রেডিকশন হয় K-টি নিকটতম প্রতিবেশীর মানের গড় (average) নিয়ে।

৮. Scikit-learn ব্যবহার করার সুবিধা

- ✓ খুব কম কোডে সম্পূর্ণ মডেল:
দূরত্ব হিসাব বা ভোটাবুটির লজিক নিজে কোড না করেই মাত্র কয়েক লাইনে ট্রেনিং সম্পন্ন করা যায়।
- ✓ দ্রুত ও অপটিমাইজড সার্চ পদ্ধতি:
`KDTree` বা `BallTree` -এর মতো ডেটা স্ট্রাকচার ব্যবহার করে নিকটতম প্রতিবেশী খোঁজা হয়, যা সরাসরি সব দূরত্ব হিসাব করার চেয়ে অনেক দ্রুত।
- ✓ সহজে K টিউন করা যায়:
`GridSearchCV` -এর মাধ্যমে সহজেই সবচেয়ে ভালো K স্বয়ংক্রিয়ভাবে বেঁধে দেওয়া যায়।
- ✓ ক্লাসিফিকেশন ও রিগ্রেসন—দুটোই সাপোর্ট করে:
`KNeighborsClassifier` ও `KNeighborsRegressor` আলাদাভাবে পাওয়া যায়।

৯. Scikit-learn ব্যবহার করার সীমাবদ্ধতা

- ✗ বড় ডেটাসেটে এখনও ধীরগতির:
অপটিমাইজড হলেও, প্রতিটি প্রেডিকশনের সময় প্রতিবেশী খোঁজার কাজ চলতেই থাকে, তাই খুব বড় ডেটাসেটে এটি অন্যান্য অ্যালগরিদমের চেয়ে ধীর।
- ✗ ফিচার স্কেলিং ভুলে গেলে ফলাফল সম্পূর্ণ ভুল হতে পারে:
sklearn নিজে থেকে স্কেলিং করে না, এটি ব্যবহারকারীকেই করতে হয়।
- ✗ মেমরি ব্যবহার বেশি:
পুরো ট্রেনিং ডেটাসেট মেমরিতে রাখা হয়, যা বড় ডেটাসেটে সমস্যা তৈরি করতে পারে।

১০. সংক্ষিপ্ত সারাংশ (Summary)

- Scikit-learn-এর `KNeighborsClassifier` (ক্লাসিফিকেশন) ও `KNeighborsRegressor` (রিগ্রেসন) ব্যবহার করে মাত্র কয়েক লাইনে KNN মডেল তৈরি করা যায়।
- মূল ওয়ার্কফ্লো: ডেটা ভাগ করা → স্কেলিং (অত্যাবশ্যক) → ট্রেনিং (`.fit()`) → প্রেডিকশন (`.predict()`) → মূল্যায়ন।
- সঠিক K বাছাই করতে Elbow Method বা `GridSearchCV` ব্যবহার করা হয়।
- `n_neighbors`, `weights`, ও `metric` -এর মতো প্যারামিটার মডেলের পারফরম্যান্সে বড় প্রভাব ফেলে।
- ফিচার স্কেলিং KNN-এর জন্য অত্যন্ত গুরুত্বপূর্ণ, কারণ মডেলটি সম্পূর্ণভাবে দূরত্বের হিসাবের উপর নির্ভরশীল।